

1. Class vs Struct vs Record in C#

Classes, Structs, and Records are used to group data and behaviors together in C#, but they have fundamental differences in their usage and internal mechanics:

A. Class

- **Data Type:** Reference Type, meaning it is stored in the Heap, while a reference to it is kept on the Stack.
- **Inheritance:** Fully supports inheritance (a class can inherit from another class).
- **Mutability:** By default, it is Mutable, meaning its property values can be modified after creation.
- **Best Use Case:** Used for large, complex models containing behavioral logic and data that changes over time (such as Student or Instructor classes in a university management system).

B. Struct

- **Data Type:** Value Type, stored directly on the Stack (or inline within a reference type if embedded), making it faster for small and simple data structures.
- **Inheritance:** Does not support inheritance (a struct cannot inherit from another struct or class, though it can implement interfaces).
- **Mutability:** Designed to be immutable (read-only after creation) to avoid unexpected side effects.
- **Best Use Case:** Used for small, lightweight data structures that represent simple values (like a coordinate point `Point(X, Y)` or custom date/time types).

C. Record

- **Data Type:** Reference Type (stored in the Heap, similar to a class).
- **Equality:** One of its main features is **Value-based Equality**, meaning it compares objects based on their actual data content rather than their memory reference.
- **Mutability:** Primarily designed to be **Immutable** by default, supporting the `with` expression for non-destructive mutation.
- **Best Use Case:** Ideal for Data Transfer Objects (DTOs) or immutable data models that do not change after initialization.

2. What is an Indexer in C#?

An **Indexer** is a powerful feature in C# that allows instances of a class or struct to be indexed **just like an array**.

- **Core Concept:** Instead of creating a custom method to retrieve items (such as `GetStudent(int index)`), an indexer allows you to use square brackets `[]` directly on the object instance.
- **Practical Use Case:** Once implemented in a container class like `StudentManager`, you can access elements cleanly using syntax such as `Student s = manager[0];`, making your code much cleaner and more intuitive.